Handbuch Lerneckenprojekt

Home-Assistant und ESP-32

Mauro Holzki & Philippe Simons

Am Standort Fischerhof der Hochschule Hannover gibt es Lernecken, welche über dem Campus verteilt sind. Diese Lernecken bieten eine Möglichkeit, um mit Kommilitonen oder alleine an der Hochschule zu lernen. Die Verteilung der Lernecken über mehrere Stockwerke und Gebäude, sowie eine hohe Auslastung zu Stoßzeiten führen jedoch dazu, dass viel Zeit mit der Suche einer leeren Lernecke verschwendet wird.
Um die Suche leichter zu machen und eine Auswertung der momentanen Auslastung zu ermöglichen, wird eine *Belegungserkennung der Lernecken* im Rahmen eines Studienprojekts vorgestellt.
Zweck des Projekts ist es, Studierenden auf einer Webseite anzuzeigen, welche Sitzplätze in welcher Lernecke, welchem Gebäude und welchem Stockwerk frei sind. Die Belegung wird pro Lernecke von einem Infrarot-Sensor erfasst, zentral gesammelt und laufend aktualisiert dargestellt. Dieses Handbuch beschreibt den Systemaufbau.



Abbildung 1: Platzierung des Belegungssensors samt ESP-32 über der Lernecke 1B.1.31.

Inhaltsverzeichnis

1 Einleitung 3
2 Hardware 3
3 Datenweg 4
4 ESP-32 Funktionsweise 4
4.1 IR-Sensor AMG8833 4
4.2 Belegungserkennung 4
5 Home Assistant Funktionsweise 6
5.1 Wo sind die Dateien 6
5.2 Die Dateien im Einzelnen 7
5.2.1 roh_schreiben.sh 7
5.2.2 sammeln.py 7
5.2.3 index.html 8
5.3 Automationen 8
5.3.1 ESP-Daten empfangen 8
5.3.2 Daten zusammenfassen 8
5.4 Configuration 8
6 Neue Lernecke hinzufügen 9
7 Wichtige Werte zum Anpassen 9
8 Fehlersuche 10
9 Einsatz von KI-Werkzeugen 11
Literaturverzeichnis 11

1 Einleitung

Ein ESP-32 (ESP) misst mit einem Infrarot-Sensor pro Lernecke die Belegung aller vier Sitzplätze und schickt sie an Home Assistant (HA). HA sammelt die Werte aller Lernecken in einer Datei, und eine öffentlich zugängliche Webseite zeigt die Plätze als grün (frei) oder rot (besetzt) an.

Grundgedanke: Zwischen dem ESP und der Webseite steht immer eine Datei (`daten.json`). Dadurch lässt sich jeder Teil einzeln ändern oder testen, ohne die anderen zu ändern.

Aufgerufen werden kann die Website unter https://hidd.christianwill.de/local/index.html beziehungsweise eine Etage unter https://hidd.christianwill.de/local/index.html#/etage/1B.1.

2 Hardware

Die benutzte Hardware beinhaltet folgende Bauteile:

- Heltec WiFi LoRa ESP-32 (V3) [1]
- AMG8833 [2]
- Kabel zur Verdrahtung
- 3D-gedrucktes Gehäuse (siehe Lernecke)

Die Hardware ist an der Lernecke 1B.1.31 montiert (oben) und kann dort genauer angesehen werden. Es können aber auch andere Optionen zur Orientierung oder Befestigung überlegt werden.

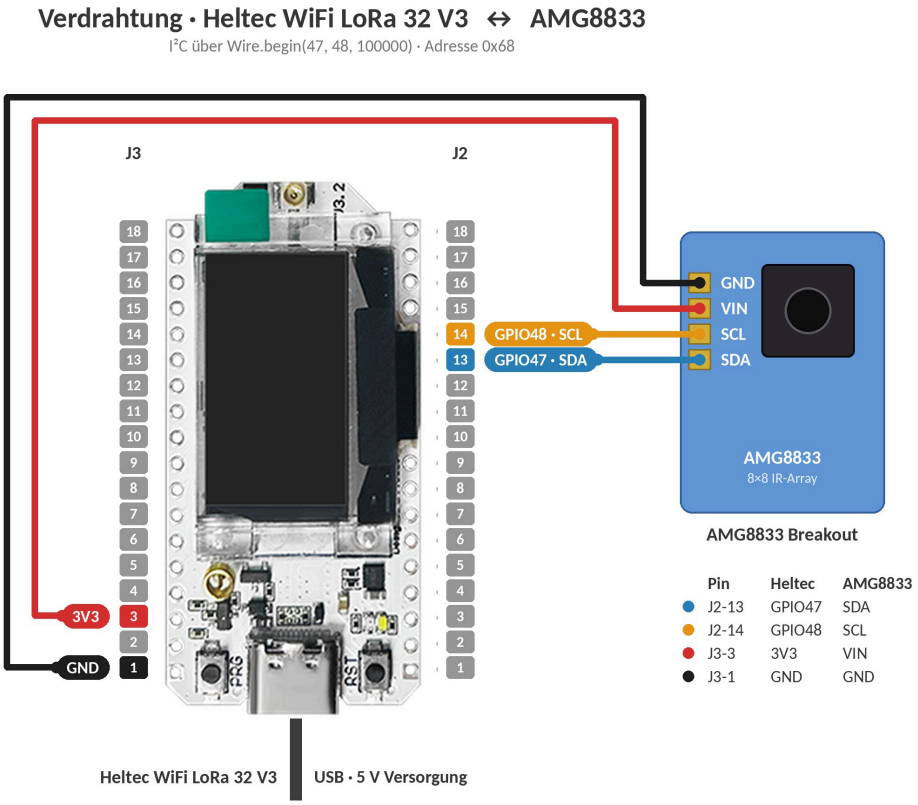


Abbildung 2: Verdrahtung des ESP32 mit dem AMG8833.

Bis jetzt ist nur diese eine Lernecke angebunden, andere Lernecken auf der Website sind nur Platzhalter.

3 Datenweg

Die Daten durchlaufen folgende Schritte:

- Der ESP sendet die Belegung einer Lernecke per HTTPS an einen Home Assistant Webhook (durch einen Cloudflare-Tunnel).
- Home Assistant nimmt die Daten an und ruft das Skript `roh_schreiben.sh` auf.
- Das Skript schreibt eine Rohdatei je Lernecke (je ESP) nach `/config/www/roh/<ID-des-ESPs>.json`.
- Nach einem Zeitintervall fasst `sammeln.py` alle Rohdateien zu einer `daten.json` zusammen.
- Die Webseite liest `daten.json` und zeigt die Belegung an.

4 ESP-32 Funktionsweise

Auf dem ESP-32 läuft ein Programm, welches Daten eines Sensors ausliest und auf Grundlage dessen die Belegung der einzelnen Plätze der Lernecken bestimmt. Dabei wird die gesamte Erkennungslogik auf dem ESP behandelt, gesendet wird nur die Platzbelegung.

4.1 IR-Sensor AMG8833

Um die Belegung einer Lernecke zu erkennen, wird ein Infrarot-Sensor benutzt, der AMG8833 von Panasonic, welcher auf einer Platine montiert ist [2]. Dieser besitzt eine Auflösung von 8x8 Pixeln, mit einem Blickwinkel von 60°.
Die Infrarot Information kann genutzt werden, um die Hitzesignatur von Menschen zu erkennen, auch deren genaue Position kann damit ermittelt werden. Trotzdem werden keine persönlichen Daten gesammelt, da die Auflösung des Sensors sehr gering ist. Als Referenz wurde eine Adafruit AMG8833 Datasheet genommen [3]. Das Auslesen der Daten ist darin beschrieben. Um die Messungenauigkeit zu verringern, wird immer ein Mittel aus mehreren Messungen des Sensors gebildet und als eine Messung behandelt.

4.2 Belegungserkennung

Die Belegungserkennung wurde in der Arduino IDE mit C++ programmiert. Das Programm beinhaltet dabei die typischen Setup() und Loop() Funktionen, Setup() wird einmalig zum Programmstart ausgeführt, Loop() immer wiederholend.
Die Logik ist insgesamt recht simpel: Die 64 Pixel des AMG8833 werden dabei in vier Zonen aufgeteilt, welche die vier möglichen Sitzplätze darstellen (vorne links, hinten links, vorne rechts, hinten rechts). Nach jeder Messung wird jeder Zone ein Zustand zugeordnet. Die drei möglichen Zustände sind:
EMPTY, CHANGING, und HUMAN.
Alle Zonen starten im Zustand EMPTY. Bei jeder Messung werden die Temperaturwerte mit der vorherigen Messung verglichen. Wenn der Unterschied größer als ein bestimmter Wert (CHANGE_DELTA = 0.5) ist, wird das Bool changed auf true gesetzt. Wenn EMPTY & changed = true, dann wird der Zustand zu CHANGING. Wenn zusätzlich die maximale Temperatur der Zone innerhalb der menschlichen Temperatur (HUMAN_MIN & HUMAN_MAX) liegt, wird der Zustand zu HUMAN, die Zone gilt als belegt. Erst, wenn über längere Zeit keine Veränderung der Temperatur mehr erkannt wurde, wird die Zone wieder zu EMPTY. Dies wird über HUMAN_HOLD (120) gesteuert. Die Logik soll dazu dienen, den Übergang einer leeren Zone (kalt, unverändert) zu einer besetzten Zone (warm, verändert) verlässlich zu erkennen.

Bei wenig Bewegung wird die Zone aber nicht direkt freigegeben, was eine richtige Belegung bei wenig Veränderung ermöglicht.

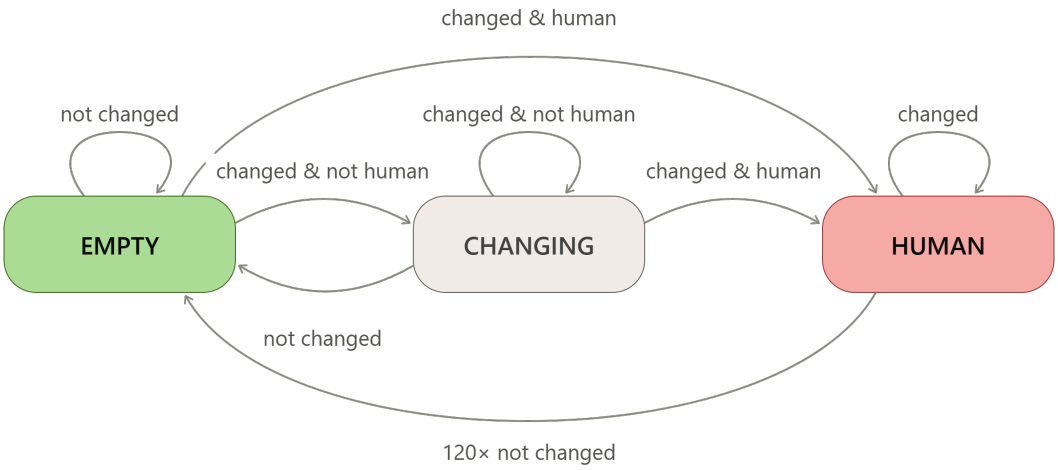


Abbildung 3: Zustandsdiagramm der Belegungserkennung

Ist eine Zone im Zustand HUMAN, gilt sie als belegt. Die Belegung der vier Zonen wird dabei in eine JSON geschrieben, belegt = 1, leer = 0, pro Platz. Diese JSON wird mit einer eigenen Funktion erstellt und alle 15 Sekunden per HTTPS-POST an die Webhook Adresse gesendet. Zusätzlich enthält sie ein Token, welches absichert, dass nur die echten ESP-Daten verarbeitet werden, sowie eine ID des ESPs, um die Lernecken zuzuordnen.

Beispiel-JSON:

```
{"token":"EINSETZEN","id":"EINSETZEN","platz1":1,"platz2":0,"platz3":1,"platz4":0}
```

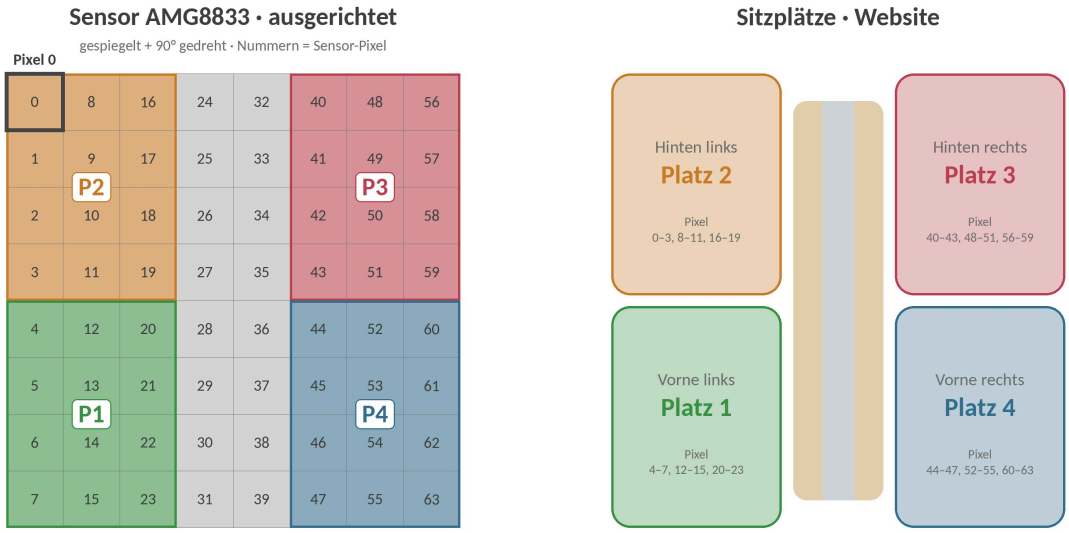


Abbildung 4: Bild zur Darstellung der relativen Orientierung des Sensors. Die gedrehte Pixelreihenfolge ergibt sich aus der Lage vom Sensor im Raum. „Vorne“ bedeutet am Gang.

5 Home Assistant Funktionsweise

Home Assistant nimmt die Daten der ESPs entgegen, legt sie ab und fasst sie zusammen. Die Skripte liegen unter `/config/lernecken/skripte`. Die öffentlichen Dateien liegen unter `/config/www` und `/config/www/roh`. WICHTIG: Diese Daten sind theoretisch für jeden einsehbar, es sollten also keine Skripte oder Ähnliches dort gespeichert werden.

5.1 Wo sind die Dateien

Datei	Zweck
`/config/lernecken/skripte/roh_schreiben.sh`	Empfängt die Daten eines ESP, schreibt eine Rohdatei
`/config/lernecken/skripte/sammeln.py`	Fasst alle Rohdateien in einem Zeitintervall zu `daten.json` zusammen
`/config/configuration.yaml`	Registriert die beiden Skripte als `shell_command`
`/config/automations.yaml`	Zwei Automationen (Empfangen, Zusammenfassen)
`/config/www/index.html`	Die Webseite
`/config/www/daten.json`	Laufzeit-Ergebnis, entsteht automatisch
`/config/www/roh/*.json`	Laufzeit-Rohdateien, entstehen automatisch

Hinweis zum Pfad: Derselbe Ordner erscheint unter zwei Namen. In Home Assistant (Config, `configuration.yaml`, File Editor) heißt er `/config/…`, im Terminal per SSH dagegen `/homeassistant/…`. In der `configuration.yaml` immer `/config/…` verwenden; legt man die Skripte per SSH ab, ist es derselbe Ort unter `/homeassistant/…`.

5.2 Die Dateien im Einzelnen

5.2.1 roh_schreiben.sh

Wird von der Automation *ESP-Daten empfangen* aufgerufen, sobald ein ESP sendet. Bekommt vom ESP: Token, ID und die Belegung mit 4 Argumenten (0 frei, 1 besetzt).

- Prüft zuerst das Token.
- Schreibt dann eine Datei unter `/config/www/roh/ID.json` mit den vier Plätzen und einem Zeitstempel. Ein neuer Aufruf derselben Lernecke überschreibt die Datei.
- Der Zeitstempel ist wichtig: `sammeln.py` rechnet damit das Alter der Daten aus.
- Die roh Datei ist für jeden öffentlich zugänglich

5.2.2 sammeln.py

Läuft in einem Zeitintervall und liest alle Rohdateien aus `/config/www/roh/`. Daraus wird eine Datei: `/config/www/daten.json`.

Wenn eine Rohdatei aus `/config/www/roh/` länger als `AUSFALL_SEK` nicht aktualisiert wurde, werden die Plätze auf null gesetzt. Die Webseite zeigt dann „keine Daten“ an, statt den alten Zustand weiter anzuzeigen.

Die Datei wird atomar geschrieben (erst in eine temporäre Datei, dann umbenennen), damit in die `daten.json` nie eine halb geschriebene Datei eingeht.

5.2.3 index.html

Der HTML-Code der Webseite. Die Struktur ist fest (Gebäude, Etage, Lernecke), die Belegung kommt per `fetch` aus `/config/www/daten.json`. Verknüpft wird über das Feld `esp` bei jeder Lernecke. Dies muss mit der ID übereinstimmen, die der ESP sendet.

Wichtige Eigenschaften:

- Struktur und Messwerte sind getrennt, eine neue Lernecke ist ein Eintrag im STRUKTUR-Block.
- Zweite Ausfall-Sicherung: Ist `daten.json` gar nicht erreichbar, zeigt die Webseite bei den Lernecken „keine Daten“.
- Jede Etage ist über die Adresse direkt aufrufbar (praktisch für QR-Codes).

5.3 Automationen

In Home Assistant laufen zwei Automationen. Sie stehen in `/config/automations.yaml` und lassen sich auch über die Oberfläche unter Einstellungen ⟶ Automationen bearbeiten.

5.3.1 ESP-Daten empfangen

Diese Automation wartet auf Daten der ESPs. Der Auslöser ist ein Webhook, eine feste Internetadresse, an die der ESP seine Werte per POST schickt:

`https://<domain>/api/webhook/<webhook_id>`

Sobald ein ESP Daten schickt, ruft die Automation das Skript `roh_schreiben.sh` auf und übergibt ihm die empfangenen Werte: Token, ID und die vier Plätze.

Drei Einstellungen sind wichtig:

- `local_only:false` muss so gesetzt sein, sonst nimmt Home Assistant den Webhook nur aus dem eigenen Netz an und der ESP von außen kommt nicht durch.
- `allowed_methods:POST` nur POST ist erlaubt. Ein Aufruf im Browser erreicht die Automation nicht.
- `mode queued` mit `max 20`: senden mehrere ESPs gleichzeitig, werden ihre Anfragen nacheinander abgearbeitet.

5.3.2 Daten zusammenfassen

Diese Automation hat keinen Auslöser, sondern läuft nach einem Zeitintervall: alle dreißig Sekunden (`time_pattern` mit `seconds: "/30"`). Sie ruft `sammeln.py` auf, das alle Rohdateien zu `daten.json` zusammenfasst.

Die beiden Automationen arbeiten unabhängig: Die erste reagiert, immer wenn ein ESP sendet. Die zweite läuft in festem Takt. Damit wird ein Ausfall erkannt. Sendet ein ESP nicht mehr, schreibt die erste Automation nichts, aber die zweite läuft weiter und setzt die betroffene Lernecke nach fünf Minuten auf „keine Daten“.

5.4 Configuration

Die `configuration.yaml` ist die Konfigurationsdatei von Home Assistant unter `/config/configuration.yaml`. Für das Projekt steht dort die Registrierung der beiden Skripte als `shell_command`.

Ein `shell_command` gibt einem Skript einen Namen, unter dem es eine Automation aufrufen kann. Ohne diesen Eintrag findet Home Assistant die Skripte nicht. Die Automation „ESP-Daten empfangen“ ruft `shell_command.esp_speichern` auf, und das verbindet diesen Namen mit der tatsächlichen Datei `roh_schreiben.sh`. Aus Sicherheitsgründen kann eine Automation nicht direkt einen shell-string zur Laufzeit ausführen, diese müssen erst registriert werden.

Die beiden Einträge:

- `esp_speichern` übergibt dem Empfangs-Skript die Werte in fester Reihenfolge: Token, ID und die vier Plätze. Die `{{ … }}` sind Platzhalter, die von der Automation gefüllt werden.
- `daten_sammeln` ruft das Sammel-Skript ohne Argumente auf.

6 Neue Lernecke hinzufügen

- ESP aufsetzen und in ESP_Lerneckenerkennung_V1.ino mit Geräte-ID und globalem Token versehen.
- In `/config/www/index.html` im Block STRUKTUR einen Eintrag ergänzen (beim richtigen Gebäude und Stockwerk). Das Feld `esp` muss der ID entsprechen, die in ESP_Lerneckenerkennung_V1.ino steht.
- In `roh_schreiben.sh` einen neuen Block mit neuem ESP und Token erstellen.
- Sitzplatzreihenfolge zur Ausrichtung des Sensors kalibrieren:
  - Plätze belegen
  - Schauen welche Plätze auf der Website angezeigt werden
  - Reihenfolge der Zonen im ESP Skript oder Reihenfolge in index.html ändern

Mehr ist nicht nötig. Die Skripte und Automationen bleiben unverändert, sie arbeiten für jede Lernecke gleich. Die bisher vergebenen IDs sind nur Platzhalter, sie können ausgetauscht werden. Es macht jedoch Sinn, sich an den Raumnummern zu orientieren.

7 Wichtige Werte zum Anpassen

Diese Werte bestimmen das Verhalten und lassen sich ändern (das ändern des ESP Skripts kann dazu führen, dass Personen nicht mehr erkannt werden):

Wert (Datei)	**Bedeutung**
Home Assistant	
`AUSFALL_SEK` (`sammeln.py`)	Nach wie vielen Sekunden ohne Kontakt „keine Daten“.
`webhook_id` (`automations.yaml`)	Teil der öffentlichen Adresse. Lang und zufällig wählen.
`ABRUF_MS` (`index.html`)	Wie oft die Webseite neu lädt (Standard 30 Sekunden).
ESP	
`AUTH_TOKEN`	Gemeinsames Passwort. Muss in `roh_schreiben.sh` gleich sein.
`DEVICE_ID`	Geräte-ID. Muss in `index.html` gleich sein.

Wert (Datei)	Bedeutung
`HUMAN_MAX, HUMAN_MIN`	Temperaturen, zwischen denen als mensch eingestuft wird
`AVG_FRAMES`	Über wie viele Messungen gemittelt wird
`CHANGE_DELTA`	Delta, ab dem ein Pixelwert als „verändert“ gilt
`HUMAN_HOLD`	Wie viele Zyklen keine Veränderung gemessen wird, bevor die Zone als leer gilt. 1 Zyklus ist Messung + Delay (~1 Sekunde).

8 Fehlersuche

Erste Anlaufstelle ist der serielle Monitor (115200 Baud). `printZoneGrid()` gibt pro Zone Max-Temp, Zustand und `hold` aus — damit ist fast jedes der folgenden Probleme direkt sichtbar.

Symptom	Ursache	Prüfen
„Sensor nicht gefunden!“ beim Start	Falsches Board/Pins oder belegte I²C-Adresse	`Wire.begin(47, 48, …)` sind ESP32-S3-Pins; Verkabelung SDA/SCL, Adresse `0x68`.
Nach Neustart kurz alles CHANGING/HUMAN	Baseline fehlgeschlagen, `prevFrame` bleibt 0	Legt sich nach 1-2 Zyklen; steht „Baseline-Frame gesetzt.“ im Log?
Webseite zeigt „keine Daten“, obwohl ESP sendet	`AUSFALL_SEK` zu knapp, vieles möglich	`AUSFALL_SEK` deutlich > `SEND_INTERVAL` (15 s) + Sammeltakt
„gesendet“, kommt aber nicht an	Token, Webhook-ID oder Automation	`POST -> <code>` im Monitor: `<0` keine Verbindung, `401` Token/ID, `2xx` ok

9 Einsatz von KI-Werkzeugen

Bei der Erstellung dieses Handbuchs und beim Programmieren wurden KI-Werkzeuge als Hilfsmittel eingesetzt. Konzeption, Ideen und die Umsetzung liegen bei den Autoren.

Werkzeug	**Einsatzzweck**	**Betroffene Stellen**
Claude (Anthropic)	Erstellung der Abbildungen (Zustandsdiagramm, Sensor-Zonen)	Abb. 1-2
Claude (Anthropic)	Fehlerbehebung im Code	Code
Claude (Anthropic)	Vorschläge zu möglichen Fehlern (Zeitzone, WLAN-Reconnect, ID-Prüfung)	Kap. Fehlersuche
Claude (Anthropic)	Typst-Formulierungshilfen	Verteilt
Claude (Anthropic)	Programmierung der HTML-Website	Code

Literaturverzeichnis

[1] Heltec Automation, „WiFi LoRa 32 (V3) — ESP32-S3 + SX1262 LoRa Node“. Zugegriffen: 2. September 2026. [Online]. Verfügbar unter: https://heltec.org/project/wifi-lora-32-v3/

[2] Seeed Studio, „Grove — Infrared Temperature Sensor Array (AMG8833)“. Zugegriffen: 2. September 2026. [Online]. Verfügbar unter: https://wiki.seeedstudio.com/Grove-Infrared_Temperature_Sensor_Array-AMG8833/

[3] Panasonic / Adafruit Industries, „Grid-EYE AMG8833 Specifications (Reference)“. Zugegriffen: 2. September 2026. [Online]. Verfügbar unter: https://cdn-learn.adafruit.com/assets/assets/000/043/261/original/Grid-EYE_SPECIFICATIONS%28Reference%29.pdf

[4] Home Assistant, „Automation Trigger — Webhook Trigger“. Zugegriffen: 2. September 2026. [Online]. Verfügbar unter: https://www.home-assistant.io/docs/automation/trigger/

[5] Home Assistant, „Shell Command Integration“. Zugegriffen: 2. September 2026. [Online]. Verfügbar unter: https://www.home-assistant.io/integrations/shell_command/

[6] Espressif Systems, „ESP32-S3 — Produktübersicht und Datenblatt“. Zugegriffen: 2. September 2026. [Online]. Verfügbar unter: https://www.espressif.com/en/products/socs/esp32-s3